

SolarPulse Analytics

Project Documentation

Renewable Energy Data Analytics & Predictive Modeling for Sustainability

Team 1 | DEPI Initiative

April 2026

1. Project Planning & Management

1.1 Project Overview & Abstract

This project delivers an integrated BI solution for solar energy optimization, utilizing a Multi-Fact Star Schema to synchronize generation and weather data. It features automated Status Normalization, an advanced custom Python integration for feature engineering, and a Power BI Machine Learning engine for real-time AC power forecasting, providing stakeholders with actionable insights into asset health, financial optimization, and sustainability KPIs.

1.2 Problem Statement

- **Production Volatility:** Solar output is unstable due to rapid weather changes, making it unreliable for direct grid dependence.
- **Planning Uncertainty:** Without accurate forecasts, it is difficult for national grids to balance load and demand dynamically.
- **Hidden Inefficiency:** Environmental factors like Module Overheating or Dust accumulation (soiling) reduce system efficiency significantly without being detected by traditional macro-monitoring software.

1.3 Project Objectives

- **AI-Driven Forecasting:** Building a robust Machine Learning model to forecast future energy yield and establish predictive output baselines based on weather inputs.
- **Anomaly Detection:** Identifying real-time performance gaps to pinpoint specialized maintenance needs and quantifying exact power drops.
- **Strategic Visualization:** Delivering a multi-page, role-based interactive Power BI dashboard to support technical, executive, and financial decision-making.

1.4 Project Timeline & Milestones (Gantt)

The project was executed over a 4-week duration aligned with the DEPI Graduate Track deadlines. Each phase was tracked rigorously using Microsoft Excel to monitor daily tasks and resource allocation.

Week	Phase	Key Activities	Start	End
Week 1	Foundations	Data Collection, SQL Database Setup, Excel Audit	01 Apr 2026	07 Apr 2026
Week 2	Engineering	Data Cleaning, SQL Joining, EDA & Feature Engineering in Python	08 Apr 2026	15 Apr 2026
Week 3	Intelligence	ML Model Training, System Loss Logic, Testing, Validation	16 Apr 2026	23 Apr 2026
Week 4	Delivery	Power BI Dashboard Design, Corporate ESG Metrics, Final Presentation	24 Apr 2026	30 Apr 2026

1.5 Task Assignment & Team Roles

Team Member	Primary Role	Key Responsibilities
Hassan Abdelzaher	Team Leader & ML Engineer	Developing Predictive Models, Python Integration & Regression Pipeline.
Mohamed Saaed	Data Analyst	Conducting EDA, identifying correlations, Heatmaps, and Feature Engineering.
Kirolos Wagdy	Data Architect	Designing Schema, ensuring Data Integrity, and managing Ingestion.
Aml Abdelaziz	BI Developer & Coordinator	Designing Power BI Layouts, DAX Measures development, and Project Documentation.

1.6 Risk Assessment & Mitigation Plan

Risk	Impact	Mitigation Strategy
Data Gaps (Missing readings)	High	Fill missing sensor values dynamically using SQL analytical partitioning and imputation rules.
Model Overfitting	Medium	Use Train-Test Split and Cross-Validation to ensure generalized operational accuracy.
Sensor Inaccuracy	Medium	Perform Outlier Detection using SQL and Python filters during the ingestion phase to eliminate noise.
Soiling & Dust Impact	High	Deploy an AI-Driven Anomaly Detection equation comparing expected vs actual production to identify cleaning thresholds.
Equipment Downtime	High	Identify failure signatures via SQL queries detecting zero-output SOURCE_KEY records during daylight hours, flagging units immediately.

1.7 Key Performance Indicators (KPIs)

- Total and Actual AC Power
- Performance_Ratio
- Daily and average Production
- Lifetime Total Energy
- Inverter Efficiency and temperature impact
- Avg Irradiation
- Max Module Temp
- Actual Operating Time Power loss
- Sustainability Metrics
- Model Accuracy
- Error Minimization
- System Health

2. Requirements Gathering

2.1 Stakeholder Analysis

Stakeholder	Primary Focus	Key Data Needs
CEO / Board	Portfolio health, sustainability goals	Total and Actual AC Power , Carbon Offset metrics
COO / Grid Operators	Grid stability, operational uptime, capacity asset use	System Availability %, peak power curves
O&M Manager	Equipment health, predictive maintenance, efficiency	Inverter health matrix, thermal correlation
Sustainability Officer	Environmental impact, regulatory compliance, ESG indices	CO2 displacement (tons), Trees Planted Equivalent, Coal Avoided

2.2 User Stories & Use Cases

Use Case 1: Predictive Performance Benchmarking

- **Actor:** Plant Manager
- **Goal:** Compare Actual AC Power against Predicted Yield to identify underperformance.
- **Process:** The manager accesses the 'AI-Insights' tab to view ML model predictions (dashed profile) against actual production data.
- **Outcome:** Clear identification of performance gaps caused by soiling or degradation rather than shifting weather conditions.

Use Case 2: Asset Health Monitoring

- **Actor:** O&M Engineer
- **Goal:** Identify underperforming inverter units to schedule targeted cleaning or hardware checks.
- **Process:** The engineer uses the Inverter Performance Matrix to look for units showing efficiency drops below the 95% safety threshold.
- **Outcome:** Proactive maintenance of specific flagged items (e.g., Inverter 1 and 12).

Use Case 3: Revenue Loss Analysis

- **Actor:** CFO / Financial Analyst
- **Goal:** Quantify energy loss and potential revenue leakage due to inverter downtime during peak hours.
- **Process:** The analyst reviews the total System Power Loss metric (currently identifying 7,586.6K total loss) and maps it against feed-in tariffs.
- **Outcome:** Data-driven maintenance budget justification and clear ROI calculation for emergency cleaning cycles.

Use Case 4: Sustainability Reporting

- **Actor:** Sustainability & Compliance Officer
- **Goal:** Generate a verifiable carbon displacement report for regulatory submission and marketing.
- **Process:** The officer accesses the 'Sustainability Dashboard' displaying total CO2 displacement and detailed green impact metrics.
- **Outcome:** Compliant regulatory ESG reports and compelling public relation material backed by real physical metrics.

2.3 Functional Requirements

ID	Feature	Description
FR-01	Data Ingestion	System shall import raw Plant Generation and Weather Sensor CSV datasets into SQL Server database.
FR-02	Data Transformation	System shall clean, normalize, and enrich data through optimized SQL views and Power Query logic.
FR-03	Star Schema Modeling	System shall implement a Multi-Fact Star Schema with conformed calendar and asset dimensions.
FR-04	ML Prediction Engine	System shall integrate a Python Linear Regression pipeline to calculate expected AC power and efficiency loss metrics.
FR-05	Interactive Layout	System shall deliver 5 role-specific dashboard tabs (Executive, Operations, Maintenance, Sustainability and AI-Insights).
FR-06	Anomaly Detection	System shall flag inverters performing below the 70% threshold as 'Needs Maintenance' with visual warning indicators.

FR-07	Advanced KPI Tracking	System shall calculate advanced environmental equivalents (Trees, Coal, SmartPhones Charged) using robust DAX measures.
-------	-----------------------	---

3. System Analysis & Design

3.1 System Architecture & ELT Pipeline

The SolarPulse architecture follows a modular Extract, Load, Transform (ELT) and analyze framework:

- **1. Extraction & Ingestion:** Raw datasets from Plant Generation and Weather Sensors (CSV/Excel format) are ingested directly into a Microsoft SQL Server environment.
- **2. SQL Transformation Layer:** Data integrity is enforced through primary/foreign keys and constraints. Optimized views are created for different time granularities (Hourly, Daily, Monthly) to maximize client query execution speed.
- **3. Python Analytics Engine:** Advanced cleaning, missing value interpolation, and target calculations are offloaded to integrated Python scripts running Linear Regression algorithms to append expected power metrics.
- **4. Visualization & Business Intelligence:** Data model is loaded into Power BI using a Multi-Fact Star Schema where advanced DAX calculations serve corporate KPIs to an executive 5-page dashboard.

3.2 Technology Stack

Layer	Technology	Purpose
Data Source	Kaggle Solar Dataset (CSV)	High-frequency raw generation & meteorological readings
Database	Microsoft SQL Server	Relational storage, integrity constraints, aggregated time-series SQL Views
Data Prep	Microsoft Excel	Static metadata mapping, initial audit scripts, project timeline tracking
Advanced Analytics	Python (pandas, scikit-learn)	Linear Regression training, predictive modeling, and System Efficiency Loss calculation
BI & Visualization	Microsoft Power BI (Dark Theme UI)	Star Schema modeling, DAX engine, multi-role interactive corporate dashboard

3.3 Advanced Data Modeling (Multi-Fact Star Schema)

To ensure high-performance analytics and lightning-fast cross-filtering, the data model was engineered using a Multi-Fact Star Schema architecture.

3.3.1 Conformed Dimensions & Fact Tables

- **Fact_Generation:** High-frequency time-series table containing DC_POWER, AC_POWER, DAILY_YIELD, TOTAL_YIELD, and SOURCE_KEY (Inverter ID) at 15-minute intervals.
- **Fact_Weather:** High-frequency time-series table containing AMBIENT_TEMPERATURE, MODULE_TEMPERATURE, and IRRADIATION.
- **Dim_Date (DateTable):** A centralized calendar dimension serving as the primary synchronization vector, allowing seamless multi-fact correlation between weather patterns and actual power yield.
- **Dim_Status:** Normalized table converting status text strings into space-optimized integers (Status_ID) with custom sort parameters (Day=1, Night=0, Fault=2).
- **Dim_Inverter:** Asset dimension containing metadata for each physical solar inverter unit using SOURCE_KEY as the unique identifier, allowing granular ranking and efficiency drift analysis.

3.4 Predictive Intelligence & Feature Engineering

The predictive intelligence layer is driven by an integrated Python script utilizing scikit-learn's Linear Regression model. It establishes a dynamic expected production profile based on irradiation and temperature, which is then used to isolate true equipment inefficiencies.

3.4.1 Python Code Implementation: Power Prediction & Accuracy

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

df = dataset.copy()
# Clean and normalize column names to uppercase
df.columns = [c.strip().upper() for c in df.columns]

try:
    # Dynamic column mapping to accommodate varied source formats
    col_ac = [c for c in df.columns if 'AC_POWER' in c][0]
    col_irr = [c for c in df.columns if 'IRRADIATION' in c][0]
    col_temp = [c for c in df.columns if 'MODULE_TEMPERATURE' in c][0]

    # Drop missing rows in training subset to guarantee mathematical precision
    train_df = df[[col_irr, col_temp, col_ac]].dropna()

    # Fit the Linear Regression model
    model = LinearRegression().fit(train_df[[col_irr, col_temp]],
    train_df[col_ac])

    # Append forecasts and calculate R2 accuracy score
    df['Predicted_AC_Power'] = model.predict(df[[col_irr,
col_temp]].fillna(0))
    df['Model_Accuracy'] = r2_score(train_df[col_ac],
```

```

model.predict(train_df[[col_irr, col_temp]])) * 100

except Exception as e:
    df['Model_Accuracy'] = f"Error: {str(e)}"

result = df

```

3.4.2 Python Code Implementation: System Efficiency Loss Isolate

```

import pandas as pd
from sklearn.linear_model import LinearRegression

df = dataset.copy()

# Select columns and filter out missing records
cols_to_use = ['IRRADIATION', 'MODULE_TEMPERATURE', 'AC_POWER']
df_clean = df[cols_to_use].dropna()

# Fit regression on clean historical subset
X = df_clean[['IRRADIATION', 'MODULE_TEMPERATURE']]
y = df_clean['AC_POWER']
model = LinearRegression().fit(X, y)

# Predict on original dataset filling null values with zero to prevent script
halt
df['Predicted_AC'] = model.predict(df[['IRRADIATION',
'MODULE_TEMPERATURE']].fillna(0))

# Isolate 'System Efficiency Loss' (Actual AC - Predicted Baseline)
df['System_Efficiency_Loss'] = df['AC_POWER'] - df['Predicted_AC']

```

3.4.3 SQL Implementation: Peak Production & Daily Analysis

```

/* Project: SolarPulse Analytics
   Purpose: Querying hourly peak profiles and daily conversion factors
*/

-- Query 1: Hourly Peak Hours Profile (For Dashboard Heatmaps)
SELECT
    DATEPART(HOUR, [DATE_TIME]) AS Production_Hour,
    AVG([AC_POWER]) AS Average_AC_Output,
    AVG([IRRADIATION]) AS Average_Sunlight,
    COUNT(*) AS Number_of_Readings
FROM
    [Plant_1_Generation_Data]
WHERE
    [AC_POWER] > 0 -- Focus exclusively on active daylight hours
GROUP BY
    DATEPART(HOUR, [DATE_TIME])
ORDER BY
    Production_Hour;

```

```
-- Query 2: Daily Conversion Metrics (For Technical Performance Curves)
SELECT
    CAST([DATE_TIME] AS DATE) AS Production_Date,
    SUM([AC_POWER]) AS Total_Daily_Yield,
    AVG([MODULE_TEMPERATURE]) AS Avg_Temp,
    AVG([IRRADIATION]) AS Avg_Irradiation
FROM
    [Plant_1_Generation_Data]
GROUP BY
    CAST([DATE_TIME] AS DATE)
ORDER BY
    Production_Date;
```

3.5 DAX Measures Architecture

The following advanced DAX measures were constructed to support the localized metrics and corporate ESG indicators visible on the Power BI layouts:

```
-- Core Generation Measures
Total DC Power (kW) = SUM(Fact_Generation[DC_POWER])
Total AC Power (kW) = SUM(Fact_Generation[AC_POWER])
Daily Yield (kWh) = SUMX(Dim_Date, [Total AC Power])
AC/DC Conversion Efficiency % = DIVIDE([Total AC Power], [Total DC Power], 0)

-- Core Weather Metrics
Average Irradiance = AVERAGE(Fact_Weather[IRRADIATION])
Max Ambient Temperature = MAX(Fact_Weather[AMBIENT_TEMPERATURE])
Module vs Ambient Delta = AVERAGE(Fact_Weather[MODULE_TEMPERATURE]) -
    AVERAGE(Fact_Weather[AMBIENT_TEMPERATURE])

-- System Health & Asset Quality Tracking
Performance Ratio (PR%) = DIVIDE([Total AC Power], [Total DC Power] * 0.75, 0)
-- Tailored to portfolio benchmarks
Total Power Loss = SUM(Fact_Generation[System_Efficiency_Loss]) -- Derived
    from Python isolation engine
Inverter Maintenance Status = IF([Performance Ratio (PR%)] < 0.95, "Needs
    Maintenance", "Operational")

-- Sustainability & Corporate ESG Measures
CO2 Offset (Tons) = [Total AC Power] * 0.0005 -- Formula optimized for carbon
    intensity mapping
Equivalent Trees Planted = [CO2 Offset (Tons)] * 0.1843
Coal Burned Avoided (Lbs) = [CO2 Offset (Tons)] * 557.4
Smartphones Charged Equivalent = [CO2 Offset (Tons)] * 82760
```

4. Dashboard Development & Design

4.1 Corporate Dashboard Structure Overview

The SolarPulse business intelligence interface is built on a highly professional, Dark-Theme user interface utilizing a standardized left-hand navigation pane. The solution features 5 distinct operational tabs providing specialized viewpoints:

4.2 Executive Portfolio Dashboard

- **Primary Audience:** CEO, Board of Directors, Carbon Trust Investors.
- **Purpose:** High-level strategic health, financial security, macro carbon metrics.
- **Key Visual Elements:** Displays a current global Performance Ratio (PR%) gauge tracking at 73.6% against a portfolio ceiling target of 75.0%. Displays Total Power Yield (21,170.1K kW), Daily Yield (53,808.0 kWh), and Total Carbon Avoided (10.6K Tons). Features an asset rank grid highlighting top-performing units (Inverter 11) and lowest-performing units (Inverter 19).

4.3 Operations Dashboard

- **Primary Audience:** Grid Operators, Plant Engineers, Chief Operating Officer (COO).
- **Purpose:** Real-time load balancing, peak capacity utilization, and instantaneous generation curves.
- **Key Visual Elements:** Highlights instant AC capacity metrics (5,683.7 kW), current average solar irradiation (0.2 kW/m²), and safe maximum operating panel temperature thresholds (65.5°C). Incorporates an hourly distribution curve plotting AC generation trends across active daylight intervals (06:00 AM to 06:00 PM).

4.4 Operations Analysis Dashboard

- **Primary Audience:** Data Scientists, Efficiency Analysts, Operational Strategists.
- **Purpose:** Algorithmic benchmarking, thermal coefficient analysis, and theoretical model validation.
- **Key Visual Elements:** Features a dual-line overlay chart ('Expected vs Actual Daily Production Profile') projecting the Python-trained Linear Regression model line against physical output to identify systemic gaps. Includes a scatter plot evaluating temperature impact on the operational Performance Ratio to map thermal degradation curves.

4.5 Maintenance & Asset Health Dashboard

- **Primary Audience:** O&M Managers, Field Technicians, Refurbishment Contractors.
- **Purpose:** Granular equipment health screening, conditional anomaly flags, and lost power quantification.
- **Key Visual Elements:** Integrates a robust conditional asset health table flagging underperforming units. Inverters falling below the safety line (e.g., Inverter 1 at 67.8% and Inverter 12 at 66.8%) are given a prominent red warning icon  and flagged as 'Needs

Maintenance'. Displays an aggregated 'Power_Loss' card quantifying 7,586.6K of lost capacity attributable directly to these identified anomalies.

4.6 Sustainability Dashboard (ESG Interface)

- **Primary Audience:** Sustainability Officers, Public Relations, ESG Compliance Reviewers.
- **Purpose:** Translating metric tons of carbon displacement into highly tangible corporate impact indicators.
- **Key Visual Elements:** Displays total saved carbon (10,585.0 Tons) and maps it to certified environmental conversions: 1,951.2 equivalent trees grown, 5.9M Lbs of raw coal burning avoided, 105.9K vehicle miles saved, and 876.4M mobile smartphones charged.

5. Testing & Quality Assurance

5.1 Data Quality Test Cases

TC#	Test Scenario	Expected Result	Status
TC-01	Check for NULL values in AC_POWER during daylight hours (IRRADIATION > 0)	Zero null records after Python interpolation loop execution.	Pass
TC-02	Verify duplicate records for single SOURCE_KEY at exact timestamps	No duplicates; unique primary key constraints enforced via SQL schema.	Pass
TC-03	Validate ambient temperature range limits (-10°C to 60°C)	Outlier readings successfully isolated and capped via data prep filters.	Pass
TC-04	Confirm referential integrity between Fact_Generation and Dim_Inverter	Referential integrity fully maintained; zero orphaned records found.	Pass
TC-05	Verify DATE_TIME format standardization to ISO 8601	All records unified to YYYY-MM-DD HH:MM:SS pattern.	Pass

5.2 DAX Measure & AI Model Validation

TC#	Measure / Component Tested	Validation Method	Status
TC-06	Performance Ratio (PR%) Metric	Cross-validated against manual Excel calculations for a sample day.	Pass

TC-07	CO2 Offset Environmental Logic	Verified against regional grid carbon factors (0.5 kg CO2/kWh baseline).	Pass
TC-08	Inverter Downtime Aggregator	Manually counted zero-output records for INV001 during daylight intervals.	Pass
TC-09	Python Model Fit Accuracy Score	R2 score extracted from training run loop; verified > 90% threshold.	Pass

6. Implementation Workflow

6.1 Data Engineering & Modeling Phase

The workflow was divided into sequential iterative pipelines to guarantee alignment across team functions:

- **Phase 1: Profiling & Audit** - Conducted first-pass audits of raw Kaggle source files in Excel to identify structural anomalies and date formatting drifts.
- **Phase 2: Relational Ingestion** - Established tables in SQL Server, implemented the Multi-Fact Star Schema relations, and compiled view scripts to support aggregated time layers.
- **Phase 3: Python Analytical Blend** - Integrated Python scripts into the data stream to train a Linear Regression model, mapping expected power boundaries and outputting an explicit 'System_Efficiency_Loss' attribute per record.
- **Phase 4: Dashboard Visualization** - Connected Power BI to the SQL database views, built out specialized measures folders, and constructed the dark-themed role-based layouts.

6.2 Version Control & Collaboration

All project materials, including database setup scripts (schema_setup.sql), Python regression source codes, and Power BI development layouts (FinalProjectStep1.pbix), were hosted inside a central corporate GitHub team repository utilizing feature branching models to prevent version conflicts.

7. User Manual & Operations Guide

7.1 Software & Hardware Requirements

- **Software:** Microsoft Power BI Desktop (January 2026 build or higher), Microsoft SQL Server (Express/Enterprise), Python 3.9+ environment with pandas and scikit-learn libraries installed.
- **Hardware:** 8 GB RAM minimum (16 GB recommended for high-frequency rendering), 2 GHz dual-core CPU, 10 GB available local disk space.

7.2 Installation & Setup Guide

Step 1: Database Provisioning - Execute provided schema scripts to build tables inside your local SQL instance. Import source CSV records using the standard SQL import wizard.

Step 2: Model Linkage - Open Power BI Desktop, connect to your database instance via SQL Server import connector, and verify that conformed schema relations mirror the ERD specifications.

Step 3: Dashboard Hydration - Launch 'FinalProjectStep1.pbix'. If prompted, adjust connection source string parameters to align with local server names. Click 'Refresh' to hydrate the visuals with live corporate data.

8. Sustainability Impact & Project Conclusion

8.1 Environmental Impact Matrix

SolarPulse Analytics directly aligns with UN Sustainable Development Goal #7 (Affordable and Clean Energy). By introducing granular Python predictive baselines, it facilitates carbon offset transparency, minimizes industrial component waste via preventative health alerts, and stabilizes national grids by predicting production curves.

8.2 Final Project Submission Information

Item	Detail
Project Title	SolarPulse Analytics — Renewable Energy Data Analytics & Predictive Modeling
Team Identification	Team 1 — DEPI Initiative Track
Implementation Start Date	01 April 2026
Final Presentation Deadline	30 April 2026
Code & Analytics Repository	GitHub — Team 1 SolarPulse Central Corporate Repository